

SoDam-Reverse-Eng Complete Usage Guide (English)

This guide is written so that anyone — even those who have never used computers, AI, or programming — can follow along.

Start from the beginning and work through it step by step. If you get stuck → [Chapter 11 FAQ](#) or [Chapter 10 Troubleshooting](#)

Table of Contents

1. [What Is This Tool? \(Plain Language\)](#)
 2. [Installation \(One Time Only\)](#)
 3. [Using SoDam Sibling Plugins Together](#)
 4. [Command Reference](#)
 5. [Starting Your First Analysis \(Step-by-Step\)](#)
 6. [Security and Data Flow Details](#)
 7. [Architecture \(Internal Structure\) Details](#)
 8. [File and Document Location Guide](#)
 9. [Advanced Features](#)
 10. [Troubleshooting and Error Handling](#)
 11. [FAQ: 25 Questions](#)
 12. [License, Copyright, and Commercial Use Details](#)
-

1. What Is This Tool? (Plain Language)

“No Jargon” Explanation

You: "What does this code do?"

AI: "This code handles the login function.

When a user enters their username and password,
it checks the database and, if correct, grants access."

That is all there is to it.

You show code to the AI, and the AI explains it in plain language. The analysis results are organized into a clean report.

Why Was It Built?

- When you cannot remember how your own old code worked
- When you need to understand code written by another developer (with their permission)
- When you are wondering “Is there a bug in this section?”
- When you want to find security vulnerabilities in your code

What Is Absolutely Not Allowed (Clearly Stated)

Cracking, DRM bypass, authentication bypass, password extraction, license bypass

These requests are refused. The AI explains why and stops the analysis. No matter how many times you ask, no method will be provided. This is the most important feature of this tool.

2. Installation (One Time Only)

You only need to do this once. After that, just launch and use it.

2-1. Check Prerequisites

Prerequisite 1: Node.js (Version 18 or higher)

Check if you already have it:

1. Press the **Windows key** (⊞)
2. Type `powershell` in the search box → click **Windows PowerShell**
3. When the black window opens, type the following and press **Enter**:

```
node --version
```

4. If you see a number like this, you are fine:

v20.19.0 ← any format like this is OK (v18 or higher)

5. If no number appears or it is lower than v18 → follow the “Installation” steps below

How to Install Node.js (if you do not have it):

1. Open a web browser (Chrome, Edge, Firefox, etc.)
2. Type `nodejs.org` in the address bar → Enter
3. Click the green “**LTS**” button (stable version)
4. When the `.msi` file downloads, double-click it
5. Keep clicking “**Next**” and then click “**Install**”
6. When installation completes, **reboot your computer** (it may not be recognized without a reboot)
7. After rebooting, reopen PowerShell and confirm with `node --version`

Tip: “Reboot” means Windows Start button → Power → Restart

Prerequisite 2: Claude Code

If you are reading this guide in Claude Code, you already have it. If not, search for “Claude Code” on the Anthropic official website and install it.

2-2. Plugin Installation (5 Steps)

Read Before Starting — Failure Cause #1

Claude Code reads commands based on the folder it was launched from.

- Wrong: Running `claude` from `C:\Users\name` (home folder)
- Right: Running `claude` from your project folder (`D:\my-project`, etc.)

Typing `cd other-folder` in the chat does NOT reload commands.

Always start Claude Code from your project folder.

Step 1: Start Claude Code from the Correct Folder

Open PowerShell and type:

```
cd D:\my-project-folder
claude
```

Replace `D:\my-project-folder` with **your actual project path**. You do not need an existing project. Any folder containing code you want to analyze works. Example: `cd D:\my-code\login-app`

Step 2: Register the Marketplace

In the Claude Code chat, type:



This is a private repository. Choose the method below that matches how you received the plugin.

Option A — Clone from GitHub (for invited repository members)



Requires a GitHub account and repository invitation. Request: `startmxk@gmail.com`

1. In PowerShell, run the clone command:

```
git clone https://github.com/sodam-ai/SoDam-Reverse-Eng.git
```

2. Note the path to the cloned folder (e.g., `C:\Users\YourName\SoDam-Reverse-Eng`)

3. In the Claude Code chat, type:

```
/plugin marketplace add C:\Users\YourName\SoDam-Reverse-Eng
```

Replace `YourName` with your actual Windows username.

Option B — Received as folder or zip file (no git required)

1. Unzip the received file to a convenient location (e.g., `C:\Users\YourName\SoDam-Reverse-Eng`)

2. In the Claude Code chat, type the actual path:

```
/plugin marketplace add C:\Users\YourName\SoDam-Reverse-Eng
```

If the path has spaces: `/plugin marketplace add "C:\My Folder\SoDam-Reverse-Eng"`

Step 3: Install the Plugin

After registering the marketplace, type:

```
/plugin install sodam-reverse@sodamreverse-marketplace
```

Or type `/plugin` → from the menu: 1. Select **Browse marketplaces** 2. Select **sodamreverse-marketplace** 3. Select **sodam-reverse** 4. Click **Install**

Step 4: Full Restart

Skipping this step will prevent commands from appearing. This step is mandatory.

Plugins are **only loaded when Claude Code starts**.

1. Type `/exit` in the chat → Enter
2. **Completely close** the PowerShell window (X button)
3. **Open a new** PowerShell window
4. Launch from **your project folder**:

```
cd D:\my-project-folder  
claude
```

Step 5: Verify Installation (Diagnostic Command)

Once Claude Code opens, type:

```
/re-ping
```

Successful response:

Pong! `/re-ping` is working.

If you see this response, the plugin loaded correctly.

2-3. Verify the 3-Layer Safety System

```
/re-selftest
```

Expected result:

```
[Layer 1] AI Refusal Rules    OK  
[Layer 2] deny-hook          OK  
[Layer 3] Integrity Check    OK
```

All three must show OK. If any show NG → [Chapter 10 Troubleshooting](#)

2-4. Register Integrity Hash (Fully Activate Layer 3)

For the Layer 3 integrity check to work properly, you need to register the hash.

- 1. Run `/re-selftest`
 - 2. Copy the 64-character string after `SHA-256 hash:` in the output
 - 3. Open `references/integrity.json` in the plugin folder
 - 4. Paste the copied hash as the value for the `"hooks/re-deny-guard.mjs"` entry
 - 5. Save the file
 - 6. Run `/re-selftest` again → confirm Layer 3 shows OK
-

2-5. Family Synergy Setup (Optional)

If SoDam-Harness is installed, run:

```
node scripts/re-inject-harness.mjs
```

A message saying “RE rules N injected successfully” means success.

3. Using SoDam Sibling Plugins Together

Overview of the 6 Siblings

SoDam consists of 6 plugins that work together as a team:

No.	Name	Role
1	SoDam-Harness	Overall safety belt (backup and central rule management)
2	SoDam-Loop	Safe control of repetitive tasks
3	SoDam-Context	CLAUDE.md health check

No.	Name	Role
4	SoDam-Agentic	Plan and report re-verification
5	SoDam-Prompt	Natural language request quality improvement
6	SoDam-Reverse	Code analysis (this plugin)

Recommended installation order: 1 through 6 in sequence

Relationship Between Each Sibling and Reverse

When Used with Harness

Harness acts as a “safety rule teacher” for Reverse. When installed, Reverse’s dangerous pattern blocking rules are shared with Harness.

Connection method:

```
node scripts/re-inject-harness.mjs
```

Effect: One hook covers rules for both plugins (no duplication).

Important: If Harness and Loop are installed, starting Claude Code from c:\Users\name (home folder) may block even normal operations. Always start from **your project folder**.

When Used with Context

Context is a tool that regularly checks whether CLAUDE.md files are correct. It automatically detects missing RE scope (defense/education/owned code only) declarations.

Connection method:

```
node scripts/re-inject-context.mjs
```

When output appears, copy and paste the command to run it.

When Used with Agentic

Agentic is a tool that re-reviews reports for non-developer readability.

How to use: After running `/re-start`, tell the AI:

"Please re-explain this report so even a beginner can understand it"

Then Agentic's easy-reviewer automatically steps in.

Check All Sibling Status

```
node scripts/check-family.mjs
```

Sample output:

SoDam 6-Sibling Status Check

```
Harness  OK - installed
Loop      OK - installed
Context   -- not installed
Agentic   OK - installed
Prompt    -- not installed
Reverse   OK (current plugin)
```

Active Synergies:

```
[Harness+Reverse] RE rule sharing    OK
[Agentic+Reverse] Re-review trigger  OK
[Context+Reverse] Scope check        -- (not connected)
```

4. Command Reference

`/re-ping` — Verify Installation

When to use: Right after installation, or when you want to check if commands are working.

Input:

```
/re-ping
```

Normal output:

Pong! `/re-ping` is working.

Abnormal output (or no response): → The plugin did not load. See [Chapter 10 Q1](#).

/re-start — Start Analysis

When to use: When you want to analyze new code.

Input format:

```
/re-start [file-path]
```

Examples:

```
/re-start src/login.js  
/re-start C:\my-project\auth\user.py  
/re-start app.js
```

Analysis flow: 1. AI confirms ownership (“Is this code yours?”) 2. AI obtains responsibility agreement (“Do you agree to defense purposes only?”) 3. Analysis begins 4. Standard report is output

How to agree:

yes <- type this

Vague answers like “probably” or “I think so” are not treated as agreement.

/re-report — Re-display Report

When to use: When you want to see a previous analysis report again.

Input:

```
/re-report
```

Output: Complete report from the most recent analysis.

/re-selftest — 3-Layer Safety Check

When to use: Right after installation, or when you want to confirm the safety system is working.

Input:

```
/re-selftest
```

Normal output:

```
[Layer 1] AI Refusal Rules      OK
[Layer 2] deny-hook             OK
[Layer 3] Integrity Check       OK
```

Abnormal output (some NG): → See [Chapter 10 Q6](#)

Commands Coming Soon

Command	Added in	Description
/re-android	Phase 2 planned	Android APK file analysis
/re-binary	Phase 3 planned	Executable file (.exe, etc.) analysis

5. Starting Your First Analysis (Step-by-Step)

Pre-Analysis Checklist

- ☐ Node.js v18 or higher is installed
 - ☐ Claude Code is running from **your project folder**
 - ☐ /re-ping returned “Pong!” response
 - ☐ /re-selftest shows all 3 as OK
 - ☐ The file to analyze is owned by you or you have permission to analyze it
-

Walkthrough: Your First Analysis

Step 1: Type in the Claude Code chat

```
/re-start src/login.js
```

(This is an example. Type your actual file path.)

Step 2: Answer the ownership confirmation question

The AI will ask:

```
Is this file/code something you created yourself or do you have authorization
to analyze it?
(yes / no)
```

If yes, type:

yes

Step 3: Answer the responsibility agreement question

The AI will ask again:

Do you agree that the analysis results will only be used for defense, learning, or code you own, and will never be used for illegal activities (cracking, bypass, key extraction)?
(yes / no)

If you agree, type:

yes

Step 4: Read the Analysis Results

When the analysis completes, a report in this format will be output:

```
# SoDam-Reverse Analysis Report

## 1. Summary
This code handles the user login function.
It receives a username and password, checks them against the database,
and if correct, creates a session to maintain the logged-in state.

## 2. Main Functions
- login() function (src/login.js:15)
  Main function coordinating the entire login process
- validateCredentials() function (src/login.js:43)
  Function that checks username/password against the database
- createSession() function (src/login.js:78)
  Function that creates a session on successful login

## 3. Findings
- Passwords appear to be compared without hashing (login.js:51)
  Security improvement needed (bcrypt recommended)

## 4. Uncertainties
- Session expiration time is not visible in the code
  Settings may exist in another file

## 5. Next Steps
- Check if session configuration exists in config.js
- Review whether password hashing is applied
```

Step 5: Check Where the Report Was Saved

Analysis results are automatically saved at:

[plugin-folder]/.sodam-re/reports/

You can always view them again with /re-report.

6. Security and Data Flow Details

Where Does My Code Data Go?

Many people worry “Will my code be leaked externally?” Here is the exact flow:

```
[1] My code (stored on my computer)
    | sent to AI for analysis
    v
[2] Claude AI (processed on Anthropic servers)
    Reads and analyzes the code
    Returns only analysis results to my computer
    | analysis results returned
    v
[3] Saved to .sodam-re/ folder on my computer
```

Important facts: - Code content is sent to Anthropic servers for analysis (Anthropic API Terms of Service apply) - Analysis results, consent records, and block logs are not sent externally - Passwords, keys, and tokens found during code analysis are shown as masked text before saving

3-Layer Safety System Detailed Explanation

Layer 1: AI Refusal Rules (SKILL.md)

Rules that prevent the AI itself from outputting dangerous content.

How it works: - When an analysis request arrives, the AI judges first - If the request is for “how to crack something,” the AI refuses - This judgment happens inside the AI

What is blocked: - Crack code generation - DRM bypass methods - Authentication bypass code - Password extraction - License bypass

Layer 2: deny-hook (re-deny-guard.mjs)

A safety system that monitors all tools Claude Code runs in real time.

How it works:

AI tries to write code

- > deny-hook monitors
- > Dangerous pattern detected?
 - No: Proceed normally
 - Yes: Immediately block and notify user

Blocked patterns (43 total): File writes and executions containing dangerous keywords related to crack, keygen, bypass, patch, etc. are blocked.

fail-closed principle: If the hook itself has an error, the result is **immediate halt** rather than “proceed.” This is “fail-closed.” Even if an error occurs, dangerous operations do not execute.

Layer 3: Integrity Check (_selftest.mjs)

Verifies that the safety files themselves have not been tampered with using SHA-256 hashes.

How it works:

- /re-selftest runs
- > Current hashes of core files are calculated
 - > Compared against stored hashes
 - Match: OK (normal)
 - Mismatch: Tampering detected (analysis halted)

What is SHA-256? The “digital fingerprint” of a file. If even one character in a file changes, the hash changes completely. This detects whether safety files have been tampered with.

Masking

Sensitive information found in code is automatically masked:

Found Pattern	Report Output
password = "abc123"	password = "(masked)"

Found Pattern	Report Output
api_key = "sk-abc..."	api_key = "(masked)"
token = "Bearer xyz..."	token = "(masked)"
SECRET_KEY = "super_secret"	SECRET_KEY = "(masked)"

10 masking patterns are registered (references/mask-patterns.json).

Analysis Result Storage Structure

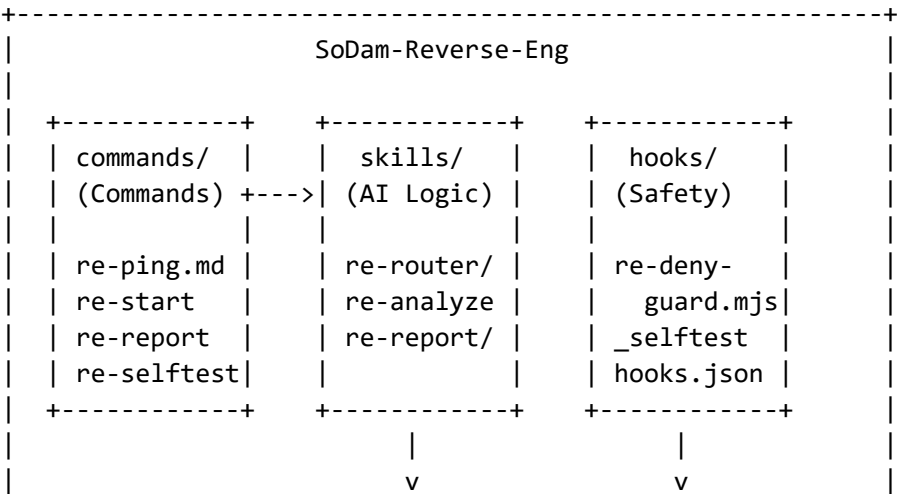
```
[plugin-folder]/
+-- .sodam-re/          <- Analysis results folder (auto-created)
    +-- reports/        <- Report files
        | +-- report-20260629-143022.md
        | +-- report-20260629-145511.md
    +-- consent/        <- Consent records
        | +-- con-20260629-143022.json
        | +-- con-20260629-145511.json
    +-- safety-log.jsonl <- Block event log
```

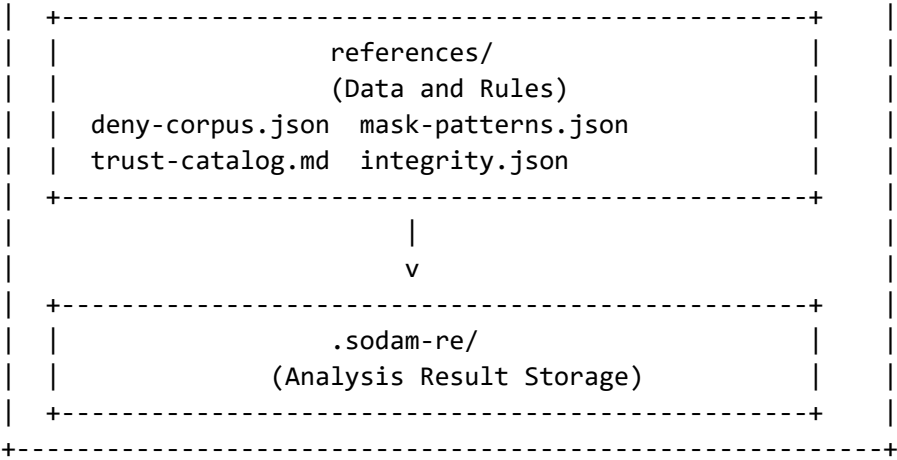
- **reports/**: Analysis reports (Markdown format)
- **consent/**: Consent records (timestamp, agreement status)
- **safety-log.jsonl**: Block event log (original content is hash-processed)

Important: The .sodam-re/ folder is registered in .gitignore. Even if you manage code with Git, **analysis results will not be uploaded.**

7. Architecture (Internal Structure) Details

Overall Structure Diagram





Component Descriptions

commands/ — Command Definitions

Defines how to behave when a user types a command like /re-start.

File	Command	Role
re-ping.md	/re-ping	Installation verification diagnostic
re-start.md	/re-start	Analysis start and consent gate
re-report.md	/re-report	Report re-display
re-selftest.md	/re-selftest	3-layer safety check

skills/ — AI Analysis Logic

Contains the AI logic where actual analysis takes place.

Folder	Role	Status
re-router/	Layer 1 safety rules and request classification	Active
re-analyze-mycode/	Source code analysis	Active
re-report/	Report generation	Active
re-analyze-android/	Android APK analysis	Phase 2 planned
re-analyze-binary/	Executable analysis	Phase 3 planned

hooks/ — Safety System

Monitors and controls all tools Claude Code runs.

File	Role
hooks.json	Hook settings (which tools to monitor)
re-deny-guard.mjs	Layer 2: real-time dangerous pattern blocking
_selftest.mjs	Layer 3: SHA-256 integrity check

`${CLAUDE_PLUGIN_ROOT}` in `hooks.json`:

`${CLAUDE_PLUGIN_ROOT}` automatically points to the folder where the plugin is installed. This means paths never need to be adjusted on any computer.

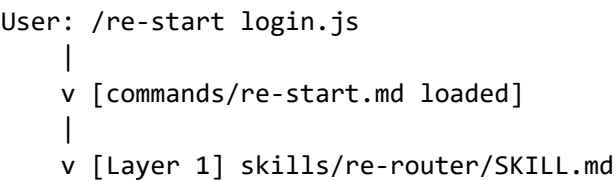
references/ — Data and Rule Files

File	Contents
deny-corpus.json	43 dangerous patterns to block
mask-patterns.json	10 sensitive information masking patterns
trust-catalog.md	15 trusted external tools
report-template.md	Standard report format
integrity.json	SHA-256 hash storage

scripts/ — Utilities

File	Function
re-inject-harness.mjs	Rule sharing with Harness
re-inject-context.mjs	Scope connection with Context
check-family.mjs	Full 6-sibling status check
check-trust-freshness.mjs	Trusted tool freshness check

Request Processing Flow (Technical Details)




```
Judges whether request is dangerous
If crack/bypass related: immediately refuse
|
v (if safe) [Consent Gate]
  Ownership confirmation question
  Responsibility agreement question
  Both "yes" required to proceed
|
v [Layer 2] hooks/re-deny-guard.mjs (parallel monitoring)
  Real-time monitoring of all tool calls
  Immediately blocks if dangerous pattern detected
|
v [Analysis] skills/re-analyze-mycode/SKILL.md
  Reads file with Read tool (never executed)
  Path validation (../ and symlinks blocked)
  Masking applied
|
v [Report Generation] skills/re-report/SKILL.md
  Organizes into standard report format
  Plain language explanation added
|
v [Storage] Saved to .sodam-re/reports/
|
v [Layer 3] Periodic integrity check (_selftest.mjs)
  Checks whether safety files were tampered with
```

8. File and Document Location Guide

Files Users Often Need

What You Are Looking For	File Location
This guide	GUIDE.en.md (current file)
Quick English overview	README.en.md
Korean overview	README.md
Korean guide	GUIDE.md
Error resolution	TROUBLESHOOTING.md
Analysis results	.sodam-re/reports/
Consent records	.sodam-re/consent/
Block log	.sodam-re/safety-log.jsonl
Full license text	LICENSE
Copyright notice	NOTICE

Files Developers Often Need

What You Are Looking For	File Location
Command definitions	commands/re-start.md, etc.
AI analysis logic	skills/re-analyze-mycode/SKILL.md
Report logic	skills/re-report/SKILL.md
Safety rules	skills/re-router/SKILL.md
Block pattern data	references/deny-corpus.json
Masking patterns	references/mask-patterns.json
Report template	references/report-template.md
Hook settings	hooks/hooks.json
Danger blocking code	hooks/re-deny-guard.mjs
Integrity check code	hooks/_selftest.mjs
Hash storage	references/integrity.json
Development progress	CHECKPOINT.md

Full Folder Tree

```
SoDam-Reverse-Eng/                                <- Plugin root folder
|
+-- .claude-plugin/                                <- Plugin declaration
|   +-- plugin.json
|   +-- marketplace.json
|
+-- commands/                                       <- 4 commands (+ 2 planned)
|   +-- re-ping.md
|   +-- re-start.md
|   +-- re-report.md
|   +-- re-selftest.md
|   +-- re-android.md                               <- Phase 2 planned
|   +-- re-binary.md                               <- Phase 3 planned
|
+-- skills/                                         <- 5 AI logic modules
|   +-- re-router/SKILL.md                         <- Layer 1 safety rules
|   +-- re-analyze-mycode/SKILL.md                 <- Source code analysis
|   +-- re-report/SKILL.md                         <- Report generation
|   +-- re-analyze-android/                       <- Phase 2 planned
|   +-- re-analyze-binary/                         <- Phase 3 planned
|
+-- hooks/                                          <- Safety system
|   +-- hooks.json
|   +-- re-deny-guard.mjs                         <- Layer 2: danger blocking
|   +-- _selftest.mjs                             <- Layer 3: integrity check
|
```

```

+-- references/                                <- Data and rules
|   +-- deny-corpus.json
|   +-- mask-patterns.json
|   +-- trust-catalog.md
|   +-- report-template.md
|   +-- integrity.json
|
+-- scripts/                                  <- Utility scripts
|   +-- re-inject-harness.mjs
|   +-- re-inject-context.mjs
|   +-- check-family.mjs
|   +-- check-trust-freshness.mjs
|
+-- samples/                                  <- Test example files
|   +-- safe-login.js
|   +-- deny-demo.txt
|
+-- .sodam-re/                                <- Analysis results (auto-created)
|
+-- README.md / README.en.md
+-- GUIDE.md / GUIDE.en.md
+-- TROUBLESHOOTING.md
+-- CHECKPOINT.md
+-- SETUP_BLOCKED_FILES.md
+-- LICENSE
+-- NOTICE

```

9. Advanced Features

Sibling Synergy Scripts

re-inject-harness.mjs

```
node scripts/re-inject-harness.mjs
```

Purpose: Register RE danger patterns in Harness safety-rules **Effect:** One hook covers both Harness and Reverse rules (no duplication) **When to run:** Once after Harness is installed

Expected output:

```

RE rules 8 (catastrophic) injected successfully
RE rules 10 (risky) injected successfully
Saved: C:\Users\name\.sodamharness\safety-rules.json

```

re-inject-context.mjs

```
node scripts/re-inject-context.mjs
```

Purpose: Add health check items so Context detects missing RE scope

Effect: Warning when CLAUDE.md does not mention

“defense/education/owned code only” **When to run:** Once after Context is installed

check-family.mjs

```
node scripts/check-family.mjs
```

Purpose: Check all 6-sibling installation status and synergy activation state at once **When to run:** Anytime (useful for diagnosing issues)

check-trust-freshness.mjs

```
node scripts/check-trust-freshness.mjs
```

Purpose: Check whether tools in the trust catalog are up to date **When to run:** Recommended quarterly

Manual Integrity Hash Update

If you legitimately modified safety files, you need to re-register the hash:

```
node hooks/_selftest.mjs
```

Copy the new SHA-256 hash from the output and update references/integrity.json.

Example Files for Testing

Normal analysis test:

```
/re-start samples/safe-login.js
```

Ordinary code without cracking or bypass. Analysis should proceed normally.

Blocking test:

```
/re-start samples/deny-demo.txt
```

File containing dangerous keywords. Analysis should be blocked.

10. Troubleshooting and Error Handling

More error resolution: [TROUBLESHOOTING.md](#)

Q1. Commands do not appear or are missing

Symptom: Typing `/re-start` or `/re-ping` produces no response, or they do not appear in autocomplete.

Cause (99% of cases): Claude Code was started from the home folder (`C:\Users\name`), or you moved folders with `cd` inside the chat.

Fix: 1. Type `/exit` in the chat, then Enter 2. Completely close the PowerShell window 3. Open a new PowerShell window 4. Start Claude Code from your project folder: `powershell cd D:\my-project-folder claude` 5. Type `/re-ping` and confirm the “Pong!” response

Q2. `/re-ping` works but `/re-start` does not

Symptom: `/re-ping` returns “Pong!” but `/re-start file-path` does not start the analysis.

Cause A — File path is wrong: The file must actually exist on your computer.

Fix: Confirm the file exists at the path you typed.

Cause B — Said “no” during consent: If you answered “no” or gave an ambiguous answer, analysis is halted.

Fix: Type `/re-start file-path` again and answer “yes” to both questions.

Q3. “Node.js not found” error

Cause: Node.js is not installed, or the computer was not rebooted after installation.

Fix: 1. Go to nodejs.org and install the LTS version 2. Reboot your computer 3. Open a new PowerShell window 4. Type `node --version` and confirm v18 or higher

Q4. Normal operations keep getting blocked (when Harness or Loop is installed)

Cause: Claude Code was started from the home folder (C:\Users\name).

Fix:

```
cd D:\my-project-folder
claude
```

Restart Claude Code from your project folder.

Q5. Consent question keeps prompting

Symptom: You said “yes” but the AI keeps asking again.

Cause: The agreement expression is too vague.

Fix: Enter one of the following clearly:

```
yes
Yes
I agree
```

Expressions like “probably,” “maybe,” or “of course” are not treated as agreement.

Q6. /re-selftest shows some items as NG

If Layer 1 shows NG:

Check whether `skills/re-router/SKILL.md` exists. If missing, copy the content from `SETUP_BLOCKED_FILES.md` and create the file.

If Layer 2 shows NG:

Check whether `hooks/re-deny-guard.mjs` exists. If missing, copy the content from `SETUP_BLOCKED_FILES.md` and create the file.

If Layer 3 shows NG (hash mismatch):

```
node hooks\_selftest.mjs
```

Copy the output hash, update `references/integrity.json`, then rerun `/re-selftest`.

Q7. Password appears in plain text in the report

Severity: High — handle immediately.

Fix: 1. Delete the problematic report file from `.sodam-re/reports/` 2. File a report at GitHub Issues

Q8. Analysis is taking too long

Fix: Analyze one file at a time:

```
/re-start src/auth.js
```

File-by-file analysis is much faster than attempting an entire folder.

Q9. “Analysis target path is outside the allowed scope” error

Fix: Use a path relative to the current folder:

```
/re-start relative-path/file.js
```

Paths containing `..` or pointing to system folders are blocked for security reasons.

Q10. Harness synergy is not connecting

```
node scripts/re-inject-harness.mjs
```

A message saying “RE rules N injected successfully” confirms success.

11. FAQ: 25 Questions

Q1. Will my code be leaked if I send it to the AI?

Code content is sent to Anthropic servers for analysis. This is how the Claude API normally operates and Anthropic Terms of Service apply. Analysis results are stored only on your computer.

Q2. Will my code be used to train the AI?

This depends on Anthropic's API Terms of Service. Check the Privacy Policy on the Anthropic official website for the current policy.

Q3. My crack request was refused. Why?

It is intentionally designed to refuse. Cracking, DRM bypass, authentication bypass, and password extraction are outside this tool's scope.

Q4. Can I use it commercially?

Yes. Commercial use is permitted under Apache License 2.0. You must include the LICENSE file and copyright notice. Check the license of the software being analyzed separately.

Q5. Is it free?

This plugin itself is free (Apache-2.0). Claude API usage fees are billed separately by Anthropic.

Q6. Can I analyze multiple files at once?

Currently, analysis is done one file at a time. Analyze multiple files in sequence.

Q7. Where are passwords found during analysis stored?

They appear in the report in masked form. The original password is not stored anywhere.

Q8. I want to delete the analysis results.

Simply delete the report files in the `.sodam-re/reports/` folder.

Q9. Can I share analysis results with others?

Confirm that masking was applied correctly before sharing. Review whether the report contains any sensitive information.

Q10. Does it work on Mac or Linux?

Phase 1 was developed with Windows as the baseline. Mac and Linux support is planned for Phases 2 and 3.

Q11. I cannot find where the report was saved.

Type `/re-report` to display the most recent report on screen. Or check the `.sodam-re/reports/` folder directly.

Q12. Does this tool execute my code?

Absolutely not. It is read-only. Your code is never executed.

Q13. What programming languages are supported?

Major languages including JavaScript, Python, Java, C/C++, Go, Rust, PHP, and TypeScript are supported. Most languages the AI understands can be analyzed.

Q14. Can APK files (Android apps) be analyzed?

Support is planned for Phase 2. Currently only source code files are supported.

Q15. Can .exe files (executables) be analyzed?

Support is planned for Phase 3. Currently only source code files are supported.

Q16. The report contains incorrect information.

AI analysis is not 100% accurate. Review the “Uncertainties” section of the report and check the code directly before making important decisions.

Q17. Can I modify and redistribute this tool?

Yes, under the Apache-2.0 license. You must document what changes you made and include the LICENSE file.

Q18. I want to suggest a new feature.

Please send your suggestion by email (startmxk@gmail.com). If you have been invited to the repository, you can also post in [GitHub Issues](#).

Q19. I found a bug.

Please report it by email (startmxk@gmail.com). Include what you typed, what result appeared, and what you expected. If you have been invited to the repository, you can also use [GitHub Issues](#).

Q20. Does it work without an internet connection?

No. An internet connection is required because Claude AI runs on Anthropic servers.

Q21. I want to remove this plugin.

```
/plugin uninstall sodam-reverse@sodamreverse-marketplace
```

Or delete the entire plugin folder and restart Claude Code.

Q22. Can it be used across multiple projects?

Yes. Once installed, the plugin works across projects. Start Claude Code from each project's folder.

Q23. Can companies use it?

Yes, Apache-2.0 permits commercial use by organizations. Check the license of the software being analyzed and Anthropic's Terms of Service separately.

Q24. Can the safety system be disabled?

No. The 3-layer safety system cannot be disabled. This is intentional by design.

Q25. Can it be used without Harness installed?

Yes. Harness is optional. Even without it, RE's own Layer 2 hook operates independently.

12. License, Copyright, and Commercial Use Details

Applied License

Apache License, Version 2.0
Copyright (c) 2026 SoDam AI Studio

Apache 2.0 is one of the most business-friendly open source licenses. It has clearer patent terms than MIT and fewer restrictions than GPL.

What Is Permitted (Explicitly Allowed)

Action	Permitted?	Conditions
Personal use	Yes	None
Modification	Yes	Must state changes made
Copying and forking	Yes	Include LICENSE and copyright notice
Redistribution (free)	Yes	Include LICENSE and copyright notice
Redistribution (paid)	Yes	Include LICENSE and copyright notice
Commercial use	Yes	Include LICENSE and copyright notice
Patent use	Yes	Terminated automatically if patent

Action	Permitted?	Conditions
Private modification	Yes	litigation filed No conditions if not distributed

Obligations (What You Must Do)

1. Include LICENSE File The LICENSE file must be included when distributing source code or binaries.

2. Preserve Copyright Notice

Copyright (c) 2026 SoDam AI Studio

This text must not be removed or modified.

3. State Changes Made If you modified the original code, state which files were modified and when.

4. Include NOTICE File If a NOTICE file exists, include it when distributing.

What Is Not Permitted

Action	Explanation
Unauthorized use of “SoDam” trademark	Cannot use as your own product name without permission
Warranty claims	Provided AS-IS — no warranty of quality or safety
Liability claims	Disclaimer applies — see full license

Commercial Use Checklist

For this plugin: - ☐ LICENSE file included - ☐ Copyright notice (c) 2026 SoDam AI Studio preserved - ☐ Changes documented (if any) - ☐ NOTICE file included

For the software being analyzed: - ☐ EULA and license of the analyzed software verified separately - ☐ Authorization to analyze confirmed (your own code or permitted)

For Claude API usage: - ☐ Anthropic Terms of Service reviewed - ☐ Commercial API usage conditions confirmed

Third-Party Trademark and Copyright Notice

Software	Owner	Official Affiliation?
Claude, Claude Code	Anthropic PBC	No
IDA Pro (Phase 3 optional)	Hex-Rays SA	No
Node.js	OpenJS Foundation	No

The above trademarks are the property of their respective owners. This plugin has no official affiliation or endorsement relationship with any of them.

Disclaimer

This software is provided for educational, defensive, and research purposes.

Users are deemed to have agreed to the following: - They are the legitimate owner of the code being analyzed or have authorization to do so - Analysis results will only be used for lawful purposes - They bear responsibility for all consequences of using this tool

The creator (SoDam AI Studio) is not responsible for: - Losses caused by misanalysis by this tool - Consequences of using this tool for illegal purposes - Service interruptions to Claude API

Full license text: [LICENSE](#) **Copyright notice:** [NOTICE](#)

*English Guide / Korean version: [GUIDE.md](#) Quick overview: [README.en.md](#)
Error resolution: [TROUBLESHOOTING.md](#)*